

Aggregation & GROUP BY

Aggregate functions take multiple rows of data as input and calculate a single result value. They're essential for summarizing and analyzing data.

GROUP BY is a SQL statement that allows us to aggregate data using aggregate functions to better understand how data is distributed per category.

Aggregate Functions

The main idea behind aggregate function is to take multiple inputs and return a single output. SQL provides a variety of aggregate functions.

[PostgreSQL: Documentation: 18: 9.21. Aggregate Functions](#)

Most common aggregate functions:

Function	Purpose	Example
<code>COUNT()</code>	Returns number of rows	<code>COUNT(*)</code> returns total rows
<code>SUM()</code>	Returns sum of all numeric values	<code>SUM(sales)</code> returns total sales
<code>AVG()</code>	Returns average value	<code>AVG(price)</code> returns average price
<code>MIN()</code>	Returns minimum value	<code>MIN(age)</code> returns youngest age
<code>MAX()</code>	Returns maximum value	<code>MAX(salary)</code> returns highest salary

Note: Aggregate function calls happen only in the **SELECT** clause or the **HAVING** clause.

Special Notes

- **AVG()** returns a floating point value many decimal places (e.g. 2.342418...)
 - You can use **ROUND()** to specify precision after the decimal.
- **COUNT()** simply returns the number of rows, which means by convention we just use **COUNT(*)**

GROUP BY

GROUP BY allows us to aggregate columns per category.

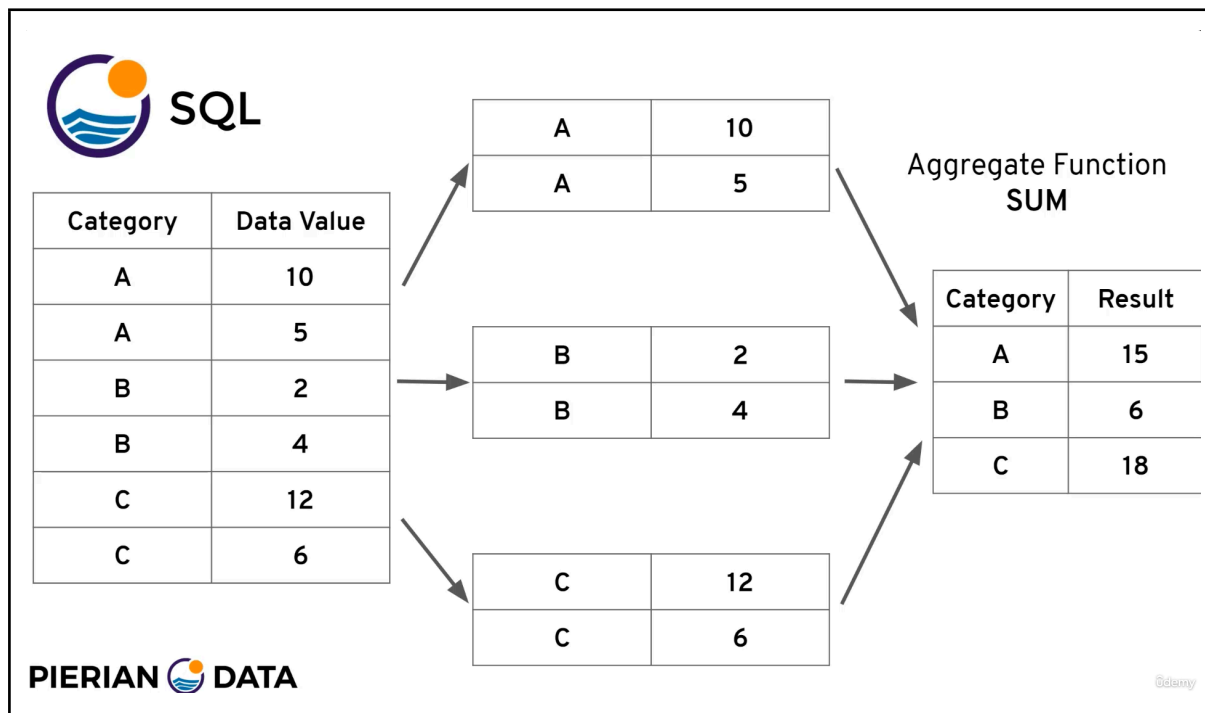
Category	Data Value
A	10
A	5
B	2
B	4
C	12
C	6

We need to choose a **categorical** column to **GROUP BY**.

Categorical columns are non-continuous.

Keep in mind, they can still be numerical, such as cabin class categories on a ship (e.g. Class 1, Class 2, Class 3)

When we use **GROUP BY**, the below happens:



General syntaxes:

```
SELECT category_col, AGG(data_col)
FROM table
GROUP BY category_col;
```

```
SELECT category_col, AGG(data_col)
FROM table
WHERE category_col != 'A'
GROUP BY category_col;
```

The **GROUP BY** clause must appear right after a **FROM** or **WHERE** statement.

*Note: In the **SELECT** statement, columns must either have an aggregate function or be in the **GROUP BY** call.*

```
SELECT category_col, AGG(data_col)
FROM table
GROUP BY category_col
```

You can also use **GROUP BY** with multiple columns:

```
SELECT company, division, SUM(sales)
FROM finance_table
GROUP BY company, division
```

*Note: **WHERE** statements should not refer to the aggregation result, We use the **HAVING** clause to filter on those results.*

```
SELECT company, SUM(sales)
FROM finance_table
GROUP BY company
ORDER BY SUM(sales)
```

If you want to sort results based on the aggregate, make sure to reference the entire function

HAVING

The **HAVING** clause allows us to filter after an aggregation has already taken place. Therefore, this clause comes only after the **GROUP BY**.

```
SELECT company, SUM(sales)
FROM finance_table
WHERE company != 'Google'
GROUP BY company
```

We've already seen we can filter before executing the **GROUP BY**, but what if we want to filter based on **SUM(sales)**?

```
SELECT company, SUM(sales)
FROM finance_table
WHERE company != 'Google'
GROUP BY company
```

We can not use **WHERE** to filter based off of aggregate results, because those happen **after** a **WHERE** is executed.

```
SELECT company, SUM(sales)
FROM finance_table
WHERE company != 'Google'
GROUP BY company
HAVING SUM(sales) > 1000
```

HAVING allows us to use the aggregate result as a filter along with a **GROUP BY**